

Embedded Linux & System Software Interview Handbook

Chapter 5 - Linux Device Drivers (Part 3)

Topics: Character Device Drivers, Major/Minor Numbers, file_operations, cdev, udev

5.6 Character Device Drivers

Character drivers transfer data as a stream of bytes. Typical examples include UART, GPIO, watchdogs and sensor interfaces. User applications access these devices through files in the /dev directory.

Communication Flow

Application → open()/read()/write()/ioctl() → VFS → file_operations → Character Driver → Hardware

Major and Minor Numbers

Every character device is identified by a major and a minor number. The major number identifies the driver, while the minor number identifies a particular device instance handled by that driver.

API	Purpose
alloc_chrdev_region()	Allocate major/minor numbers
cdev_init()	Initialize cdev object
cdev_add()	Register character device
class_create()	Create device class
device_create()	Create /dev node

The file_operations Structure

The kernel dispatches user-space requests through struct file_operations. Each function pointer implements one operation supported by the driver.

```
open()      release()
read()      write()
unlocked_ioctl()
poll()
mmap()
```

Typical Driver Initialization Sequence

1. Allocate device numbers.
2. Initialize cdev.
3. Register cdev.
4. Create device class.
5. Create /dev entry.
6. Driver ready for user applications.

How /dev Nodes Are Created

Modern Linux systems use udev to automatically create device nodes after the driver registers with the kernel. Earlier systems often required manual mknod commands.

Minimal Character Driver Skeleton

```
static const struct file_operations fops = {
    .owner = THIS_MODULE,
    .open = my_open,
    .read = my_read,
    .write = my_write,
    .release = my_release,
};
```

Common Debugging Tips

- Check dmesg after loading the driver.
- Verify the /dev node exists.
- Confirm major/minor numbers.
- Ensure probe() completed successfully for platform drivers.
- Validate permissions before testing from user space.

Interview Questions

1. What is a character device driver?
2. Explain major and minor numbers.
3. What is struct file_operations?
4. How does a /dev node get created?
5. Difference between read() and ioctl()?
6. Why is cdev required?

Key Takeaways

Character drivers form the foundation of Linux driver development. Understanding device registration, file_operations and user-space interaction is essential for embedded Linux and BSP interviews.